

Google Android NDK ASharedMemory Zero-Copy Architecture Specification

1. Executive Summary & Purpose

In high-throughput multi-agent and local LLM architectures on Android (such as running Qwen 3.5 or Gemma 4 on the Snapdragon 8 Elite Hexagon NPU), standard Android Inter-Process Communication (IPC) mechanisms fail:

- **Binder Transaction Buffer Limits:** Standard Android Binder limits transactions to a global 1MB pool shared across all ongoing transactions in the process. Attempting to pass tensor arrays or large context buffers over Binder triggers `TransactionTooLargeException`.
- **CPU Memory Copying Tax:** Passing large buffers over local network sockets or standard UNIX pipes forces the CPU to serialize and copy data between kernel and user spaces multiple times, degrading battery life and frame rates.

By utilizing the Android NDK `ASharedMemory` API combined with POSIX `mmap`, processes allocate anonymous shared memory backed by Linux file descriptors. Data is written once into shared RAM, and child daemons (such as `:qairt_engine` and `:llamacpp_engine`) read directly from the identical physical memory address space with zero CPU copy overhead.

2. API Contract & Lifecycle Sequence

Phase 1: Shared Memory Allocation (NDK)

The initiating daemon (e.g. `Aesc Terminal Daemon`) allocates an anonymous shared memory region backed by the Linux kernel:

```
#include <android/sharedmem.h>
```

```
#include <sys/mman.h>
```

```
#include <unistd.h>
```

```
int fd = ASharedMemory_create("llm_tensor_buffer", buffer_size);
```

```
if (fd < 0) {  
  
    // Handle allocation failure (e.g. system out of memory)  
  
    return -1;  
  
}
```

Phase 2: Mapping into Virtual Address Space

The allocator maps the descriptor into its local virtual memory:

```
void* buffer = mmap(NULL, buffer_size, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);  
  
if (buffer == MAP_FAILED) {  
  
    close(fd);  
  
    return -1;  
  
}
```

// Populate the buffer natively (e.g., token matrices, prompt context)

```
memcpy(buffer, source_tensor_data, buffer_size);
```

Phase 3: Security & Protection Lock

To prevent recipient daemons from mutating or corrupting active model weights or shared context buffers, the write capability can be permanently dropped:

// Restricts all future mappings and child operations to READ-ONLY

```
int res = ASharedMemory_setProt(fd, PROT_READ);
```

```
if (res < 0) {  
  
    // Handle protection error  
  
}
```

Phase 4: Inter-Process Handshake

The pristine file descriptor (`fd`) is transmitted across daemon process boundaries over a local UNIX domain socket using POSIX ancillary messages (`SCM_RIGHTS`).

Phase 5: Receiver Mapping & Lifecycle Teardown

1. The receiving engine receives the duplicated file descriptor via `recvmsg()`.
 2. The engine calls `mmap(NULL, buffer_size, PROT_READ, MAP_SHARED, received_fd, 0)`.
 3. The Qualcomm NPU / Hexagon Direct SDK consumes the memory buffer directly from `local_buffer`.
 4. When processing completes:
 - `munmap(buffer, buffer_size)`
 - `close(fd)` (Kernel automatically reclaims the physical pages once all file descriptors referencing the anonymous inode are closed).
-

3. Best Practices & Hardware Boundary Constraints

1. **Cache Coherency:** Always ensure cache lines are flushed if non-coherent DMA buffers are shared between CPU and Hexagon DSP/NPU.
2. **Page Alignment:** Allocate buffer sizes aligned to 4KB page boundaries (or 64KB for large tensor blocks) to optimize TLB cache hit rates.
3. **Descriptor Hygiene:** Explicitly close received file descriptors immediately after `mmap` to avoid leaking kernel file table entries.